31.2 配置类型系统详解

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~170k tokens，输出 ~7k tokens（本节）

OpenClaw 的配置类型系统是整个项目中最庞大的类型定义集合——`src/config/types.ts` 作为桶文件（barrel file），重新导出了 30 余个子类型模块。本节梳理其核心结构，帮助你理解配置对象的全貌。

31.2.1 核心类型概览（`src/config/types.ts`）

模块化拆分

`types.ts` 本身只有 32 行——它是一个纯粹的重新导出文件：

```
// src/config/types.ts
export * from "./types.agent-defaults.js";
export * from "./types.agents.js";
export * from "./types.approvals.js";
export * from "./types.auth.js";
export * from "./types.base.js";
export * from "./types.browser.js";
export * from "./types.channels.js";
export * from "./types.discord.js";
// ... 共 30+ 个子模块
export * from "./types.memory.js";
```

这种拆分策略遵循了"编辑局部性（edit locality）"原则——修改 Telegram 配置类型只需编辑 `types.telegram.ts`，不会触发其他 20+ 个模块的重新编译。

OpenClawConfig 的顶层结构

`OpenClawConfig` 是所有配置的根类型（在 `types.openclaw.ts` 中定义，通过 barrel 导出）。下表列出了它的主要顶级键：

键	类型模块	描述
`gateway`	`types.gateway.ts`	Gateway 服务器设置（端口、认证、远程访问）
`agents`	`types.agents.ts`	Agent 列表与默认配置
`models`	`types.models.ts`	模型提供者与模型定义
`tools`	`types.tools.ts`	工具配置（bash、浏览器、媒体理解等）
`channels`	`types.channels.ts`	各通道配置（WhatsApp、Telegram 等）
`hooks`	`types.hooks.ts`	钩子系统配置
`skills`	`types.skills.ts`	技能系统配置
`plugins`	`types.plugins.ts`	插件系统配置

键	类型模块	描述
session	types.base.ts	会话管理配置
messages	types.messages.ts	消息处理配置
logging	types.base.ts	日志配置
env	—	环境变量定义
meta	—	版本戳记、元数据

31.2.2 Agent 配置（types.agents.ts / types.agent-defaults.ts）

双层配置结构

Agent 配置采用**默认值 + 实例覆盖**的双层结构：

```
// types.agents.ts
export type AgentsConfig = {
  defaults?: AgentDefaultsConfig;   // 全局默认值
  list?: AgentConfig[];             // Agent 实例列表
};
```

`AgentDefaultsConfig` 定义了所有 Agent 共享的默认配置——心跳设置、模型选择、上下文压缩策略、沙箱行为等。每个 `AgentConfig` 实例可以选择性地覆盖这些默认值。

AgentConfig 关键字段

```
export type AgentConfig = {
  id: string;                       // 唯一标识
  default?: boolean;                // 是否为默认 Agent
  name?: string;                    // 显示名称
  workspace?: string;               // 工作区目录
  agentDir?: string;                // Agent 专属目录
  model?: AgentModelConfig;         // 模型配置（字符串或 primary+fallbacks 对象）
  skills?: string[];                // 技能白名单
  identity?: IdentityConfig;        // 身份配置（名称、主题色、头像）
  sandbox?: {                       // 沙箱设置
    mode?: "off" | "non-main" | "all";
    workspaceAccess?: "none" | "ro" | "rw";
    scope?: "session" | "agent" | "shared";
    docker?: SandboxDockerSettings;
  };
  tools?: AgentToolsConfig;         // 工具覆盖
  subagents?: {                     // 子 Agent 配置
    allowAgents?: string[];
    model?: string | { primary?; fallbacks? };
  };
};
```

模型配置 `AgentModelConfig` 支持两种写法：

```
// 简写：直接指定模型 ID
{ "model": "anthropic/claude-sonnet-4-20250514" }

// 完整写法：指定主模型和备用模型
{
  "model": {
    "primary": "anthropic/claude-sonnet-4-20250514",
    "fallbacks": ["openai/gpt-4o", "google/gemini-2.5-pro"]
  }
}
```

默认配置中的精细控制

`AgentDefaultsConfig` 中有几个特别值得注意的嵌套配置：

上下文压缩（Context Pruning）——当对话变长时，自动裁剪旧的工具调用结果：

```
export type AgentContextPruningConfig = {
  mode?: "off" | "cache-ttl";
  ttl?: string;                    // 缓存过期时间
  keepLastAssistants?: number;      // 保留最近 N 条助手消息
  softTrimRatio?: number;           // 软裁剪阈值（如 0.7 = 70%）
  hardClearRatio?: number;          // 硬清除阈值
  minPrunableToolChars?: number;    // 最小可裁剪字符数
};
```

> **v2026.3.9 新增** —— 上下文压缩（Compaction）相关配置也有两个新字段：
>
> ```
> // types.agent-defaults.ts 中的 CompactionConfig
> export type CompactionConfig = {
>  postCompactionSections?: string[];  // 压缩后保留的段落标题
>  // 默认 ["Session Startup", "Red Lines"]
>  model?: string;                     // 用于压缩的模型（可选，覆盖默认模型）
> };
> ```
>
> `postCompactionSections` 指定压缩后必须保留的上下文段落——比如「会话启动信息」和「红线规则」不应在压缩中丢失。`model` 允许为压缩任务单独指定一个模型（例如用更便宜的模型做压缩摘要）。

CLI 后端（CLI Backend）——允许用 CLI 命令替代内置的模型提供者：

```
export type CliBackendConfig = {
  command: string;                  // CLI 命令
  args?: string[];                  // 参数
  output?: "json" | "text" | "jsonl";
  input?: "arg" | "stdin";
  modelArg?: string;                // 模型参数标志（如 --model）
  sessionArgs?: string[];            // 会话恢复参数
};
```

31.2.3 通道配置（`types.channels.ts` / `types.discord.ts` /

`types.telegram.ts` 等）

每个通道有独立的配置类型，但共享一些通用模式：

通道	类型文件	关键配置项
WhatsApp	`types.whatsapp.ts`	`web` （WebSocket）、 `allowFrom` 、 `groups`
Telegram	`types.telegram.ts`	`botToken` 、 `webhookSecret` 、 `groups`
Discord	`types.discord.ts`	`botToken` 、 `guildId` 、 `threadMode`
Slack	`types.slack.ts`	`botToken` 、 `appToken` 、 `socketMode`
Signal	`types.signal.ts`	`signalCliPath` 、 `attachmentsDir`
iMessage	`types.imessage.ts`	`dbPath` 、 `pollInterval`
MS Teams	`types.msteams.ts`	`appId` 、 `appSecret` 、 `tenantId`
Google Chat	`types.googlechat.ts`	`credentials` 、 `projectId`

通道配置的共同模式包括：

- **DM 策略**： `pairing` （需要配对）/ `open` （开放）/ `allowlist` （白名单）
- **群组策略**：每个群组可以独立配置是否需要 @提及、回复模式、会话范围
- **发送策略**：控制哪些通道/聊天类型允许 Agent 主动发消息

v2026.3.9 为多个通道增加了新的配置项：

配置项	通道	类型/默认值	说明
`discord.autoArchiveDuration`	Discord	`60 | 1440 | 4320 | 10080` 分钟	线程自动归档的超时时间
`channels.slack.typingReaction`	Slack	`string` （emoji 名称）	Agent 处理消息时添加的 Reaction 表情
`talk.silenceTimeoutMs`	Talk（语音）	`number` （毫秒）	Talk 模式下检测到静默后发送转录的等待时长
`acp.provenance`	ACP	`"off" | "meta" | "meta+receipt"`	ACP 会话的来源追踪级别（见第 32 章）

31.2.4 安全配置（ `types.sandbox.ts` / `types.auth.ts` ）

沙箱配置

```
// types.sandbox.ts（部分）
export type SandboxDockerSettings = {
  image?: string;              // Docker 镜像名
  network?: string;            // 网络模式
  memory?: string;             // 内存限制
  cpus?: number;               // CPU 限制
  extraMounts?: string[];      // 额外挂载卷
  extraArgs?: string[];        // 额外 Docker 参数
};
```

沙箱模式有三种层级：

- `"off"` — 不使用沙箱
- `"non-main"` — 仅非主会话使用沙箱（默认推荐）
- `"all"` — 所有会话都在沙箱中运行

认证配置

```
// types.auth.ts
export type GatewayAuthConfig = {
  token?: string;              // Gateway 访问 Token
  password?: string;           // Web 控制台密码
};
```

31.2.5 模型配置（ types.models.ts ）

模型配置的设计极为灵活——用户可以自定义模型提供者，甚至接入私有部署的 API：

```
export type ModelProviderConfig = {
  baseUrl: string;                    // API 端点
  apiKey?: string;                    // API 密钥
  auth?: "api-key" | "aws-sdk" | "oauth" | "token";
  api?: ModelApi;                     // API 协议
  headers?: Record<string, string>;   // 自定义请求头
  models: ModelDefinitionConfig[];    // 模型定义列表
};

export type ModelDefinitionConfig = {
  id: string;                         // 模型 ID
  name: string;                       // 显示名称
  api?: ModelApi;                     // 覆盖提供者级别的 API 协议
  reasoning: boolean;                 // 是否支持推理模式
  input: Array<"text" | "image">;     // 支持的输入类型
  cost: {                             // 成本信息（每百万 token）
    input: number;
    output: number;
    cacheRead: number;
    cacheWrite: number;
  };
  contextWindow: number;              // 上下文窗口大小
  maxTokens: number;                  // 最大输出 token
};
```

支持的 API 协议包括：

API 标识	对应服务
`openai-completions`	OpenAI Chat Completions
`openai-responses`	OpenAI Responses API
`anthropic-messages`	Anthropic Messages API

API 标识	对应服务
`google-generative-ai`	Google Gemini
`github-copilot`	GitHub Copilot
`bedrock-converse-stream`	AWS Bedrock

`ModelsConfig` 还支持 `mode` 字段——`"merge"` 将自定义模型与内置模型合并，`"replace"` 则完全替换内置模型列表。

31.2.6 工具/技能/钩子配置（`types.tools.ts` / `types.skills.ts` / `types.hooks.ts`）

工具配置

工具配置控制 Agent 可以使用的工具及其行为。最重要的几个子配置：

- **exec**：bash 工具的执行限制（超时、工作目录、禁止命令）
- **media**：媒体处理配置（音频转录模型、图像理解、视频理解）
- **browser**：浏览器控制配置（是否启用、evaluate 权限）
- **agentToAgent**：跨 Agent 通信配置

v2026.3.9 新增了 `browser.relayBindHost` 配置项（默认 `"127.0.0.1"`），控制浏览器扩展中继服务器（Extension Relay Server）绑定的网络地址。在容器化部署中，可能需要将其设置为 `"0.0.0.0"` 以允许外部访问。

技能配置

```typescript
export type SkillsConfig = {
  allowBundled?: string[];                // 内置技能白名单
  load?: {
    extraDirs?: string[];                 // 额外技能目录
    watch?: boolean;                      // 文件变化自动刷新
    watchDebounceMs?: number;             // 刷新防抖
  };
  install?: {
    preferBrew?: boolean;                 // 优先使用 Homebrew 安装
    nodeManager?: "npm" | "pnpm" | "yarn" | "bun";
  };
  entries?: Record<string, SkillConfig>; // 每个技能的配置
};
```

钩子配置

钩子配置分为两层：外层控制 HTTP 钩子端点，内层控制内部事件钩子。

```typescript
export type HooksConfig = {
  // HTTP 钩子端点
  enabled?: boolean;
  path?: string;                          // 端点路径（默认 /hooks）
  token?: string;                         // 鉴权 Token
  presets?: string[];                     // 预设映射（如 ["gmail"]）
  mappings?: HookMappingConfig[];         // 自定义映射规则
  gmail?: HooksGmailConfig;              // Gmail 集成专用配置

  // 内部事件钩子
  internal?: {
    enabled?: boolean;
    handlers?: InternalHookHandlerConfig[];  // 遗留处理器
    entries?: Record<string, HookConfig>;    // 每个钩子的配置
    load?: { extraDirs?: string[] };         // 额外钩子目录
    installs?: Record<string, HookInstallRecord>;  // 安装记录
  };
};
```

每个钩子条目 `HookConfig` 支持 `enabled` （启用/禁用）和 `env` （环境变量注入），还可以有任意额外字段供钩子处理器读取（如 `session-memory` 的 `messages` 字段）。

本节小结

1. **类型系统**采用模块化拆分策略，30+ 个子类型文件通过桶文件统一导出，兼顾了类型完整性和编辑局部性。
2. **Agent 配置**采用默认值 + 实例覆盖的双层结构，支持灵活的模型选择（主模型 + 备用列表）、沙箱隔离、技能白名单等精细控制。
3. **通道配置**为每个消息通道提供独立的类型定义，同时共享 DM 策略、群组策略、发送策略等通用模式。
4. **模型配置**支持自定义提供者和 6 种 API 协议，允许接入任何兼容的 LLM 服务，包括私有部署。
5. **工具/技能/钩子配置**各自独立又相互关联——它们共同定义了 Agent 的能力边界和行为规则。